Blockchain Based Decentralized Medical Record Management Using Ethereum Smart Contracts.

S.Hruthik – 21BCE1960

School of Computer Science,

Vellore Institute of technology,
Chennai, India
hruthik.mani23@gmail.com

***Abstract*—The proliferation of electronic health records (EHRs) has led to challenges in managing, accessing, and securing sensitive patient data. Blockchain technology, with its inherent properties of immutability, decentralization, and transparency, offers a promising solution for enhancing medical record management. This research article proposes a system for medical record management leveraging Ethereum smart contracts, ensuring data integrity, accessibility, and privacy. The system architecture, proposed algorithm, and performance analysis demonstrate the feasibility and benefits of employing blockchain technology in healthcare.**

Keywords—Blockchain, Ethereum, Smart Contracts, Medical Records, Healthcare, Data Management, Electronic Health Record (EHR).InterPlanetary File System (IPFS).

Introduction

The digitization of medical records has reshaped healthcare delivery, yet it's accompanied by challenges such as data security and interoperability. Centralized storage systems are vulnerable to breaches and lack transparency, raising privacy concerns. In response, blockchain technology emerges as a viable solution, promising decentralized and secure medical record management. This research introduces a blockchain-based system employing Ethereum smart contracts, aiming to address EHR system shortcomings and bolster data security in healthcare.

I. Literatur Review

A. *Attaran (2022) emphasizes the pivotal role of blockchain technology in healthcare, highlighting its potential to address challenges in accessing and managing electronic health records (EHRs) securely. The study underscores the need for patients to manage their health data globally and securely share it with healthcare providers, especially during public health crises like the COVID-19 pandemic.*

B. *Jabbar et al. (2020) propose a blockchain framework, BiiMED, to enhance data interoperability and integrity in EHR-sharing systems. Their work focuses on facilitating secure exchange of EHRs among medical providers and ensuring data integrity through decentralized auditing mechanisms.*

C. *Tanwar, Parekh, & Evans (2020) explore the transformative potential of blockchain in healthcare, particularly in Healthcare 4.0 applications. Their study demonstrates how blockchain can streamline health record management, insurance billing, and smart contracts, leading to improved access to patient data and enhanced healthcare services. They leverage frameworks such as Hyperledger Fabric and Composer, along with innovative access control policies, to optimize performance metrics like latency and throughput.*

D. *Together, these studies underscore the growing recognition of blockchain's significance in revolutionizing healthcare data management. By addressing challenges related to data security, interoperability, and accessibility, blockchain technology offers a promising avenue for enhancing patient care and transforming the healthcare landscape.*

II. Proposed Work

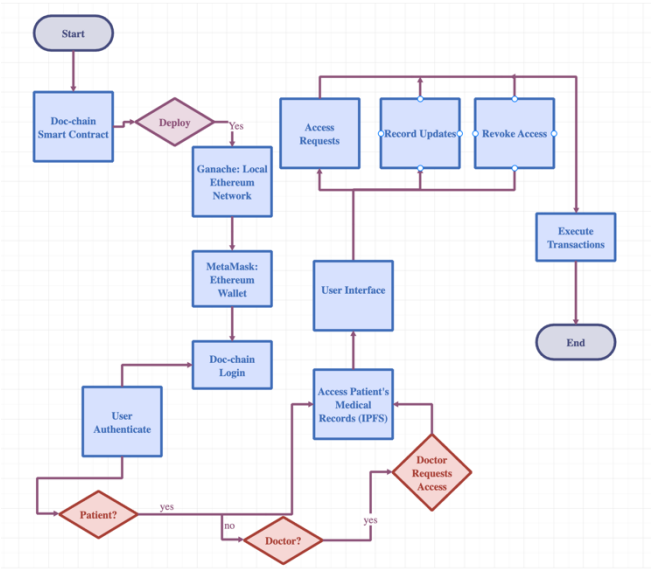
The proposed system comprises a decentralized network utilizing Ethereum smart contracts for secure medical record storage. The system architecture integrates three core components: the Ethereum blockchain network, smart contracts, and user interfaces. we ensures that patients maintain full control over their medical records by empowering them to revoke access at any time, thereby enhancing data privacy and patient autonomy. Through the Doc-chain interface, patients can initiate the revocation process with a simple click, triggering a smart contract function that invalidates the access permissions granted to the respective doctor. This functionality provides patients with the flexibility to manage their data sharing preferences dynamically and revoke access if they no longer wish to share their medical records with a particular doctor. The revocation action is executed securely through the blockchain network, ensuring that access privileges are revoked promptly and transparently. By integrating these features into the Doc-chain platform, the proposed work aims to enhance user control, privacy, and security, fostering a trusted environment for decentralized medical records management.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

III. System Architecture

The system architecture is designed to seamlessly integrate blockchain technology using Ganache, a personal blockchain for Ethereum development, and Metamask, a popular Ethereum wallet extension. Initially, Ganache is utilized to run a local blockchain network, providing a sandbox environment for testing and development purposes. Metamask, acting as the gateway to the Ethereum blockchain, is integrated with Doc-chain login, allowing users to authenticate and access the platform securely. Upon login, patients are connected to their Metamask wallet, which serves as their digital identity and grants them access to their medical records stored on IPFS. Similarly, doctors authenticate themselves through Metamask, enabling them to request access to patient records and interact with the system's functionalities. Transactions initiated by users, such as access requests, record updates, and revocation actions, are seamlessly executed through Metamask, leveraging its secure communication with the Ethereum blockchain.

The Doc-chain system architecture comprises four main components: user interfaces, IPFS, Doc-chain smart contracts, and the blockchain network.



A. Doc-chain Smart Contracts

1. Patient Smart Contracts: Manage patient information, access control lists, and interactions with IPFS.
2. Doctor Smart Contracts: Facilitate access requests, insurance claims, and interactions with patient records.

B. IPFS (InterPlanetary File System)

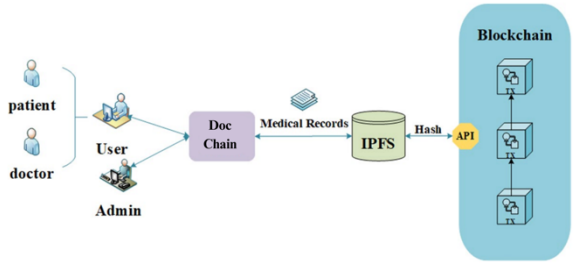
Serves as decentralized storage for medical records. Encrypts and stores patient records as files, ensuring data availability and redundancy across the network.

C. Blockchain Network (Ethereum)

Deployed to execute smart contracts and record transactions related to access requests, insurance claims, and record updates.

D. User Interfaces

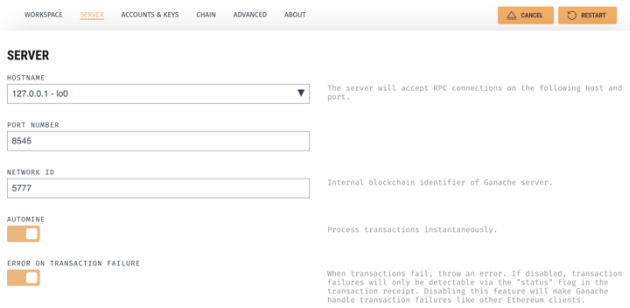
1. Patient Interface: Allows patients to register, upload, and manage their medical records securely.
2. Doctor Interface: Enables doctors to request access to patient records, view medical history, and diagnose conditions.



IV. Methodology

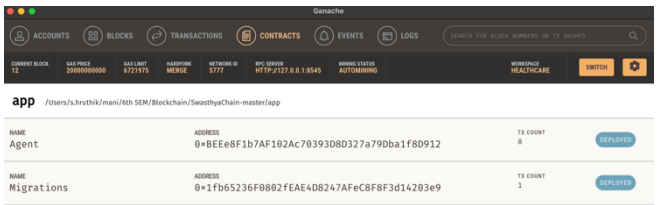
A. Setting up Development Environment :

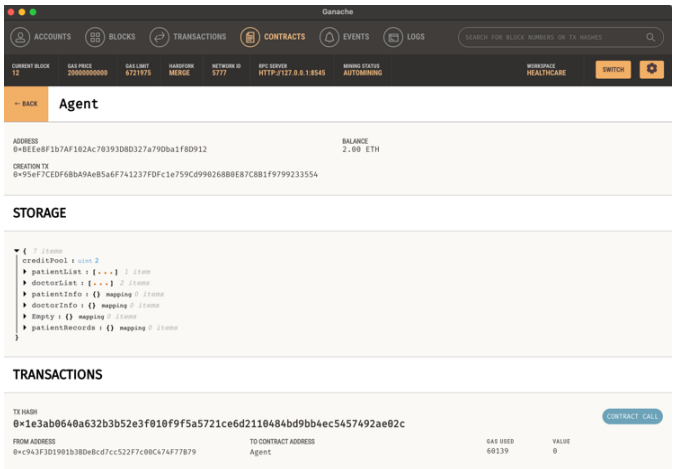
Open Ganache and click on settings in the top right corner.
Under Server tab:
Set Hostname to 127.0.0.1 -lo
Set Port Number to 8545
Enable Auto mine
Under Accounts & Keys tab:
Enable Autogenerate HD Mnemonic



B. Smart Contract Deployement :

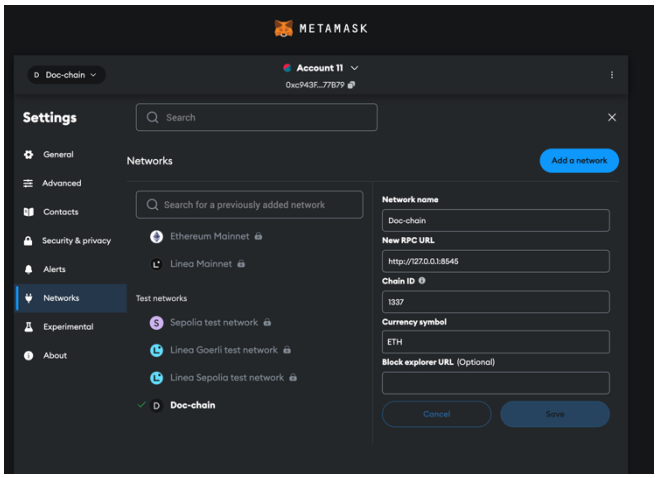
Use *truffle Migrate* to Deploy the smart Contracts to Ganache



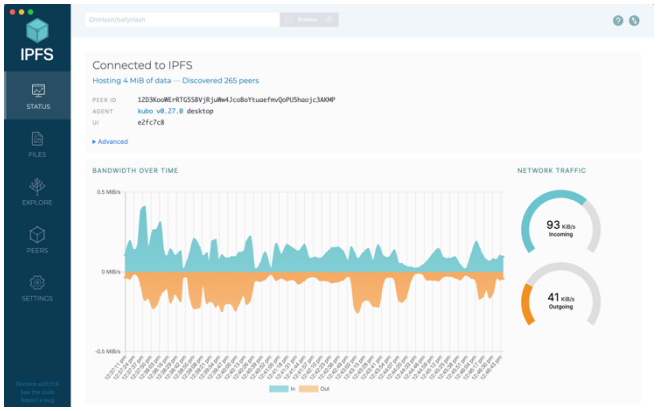


C. Integration with Metamask :

A. Import Network using RPC URL:

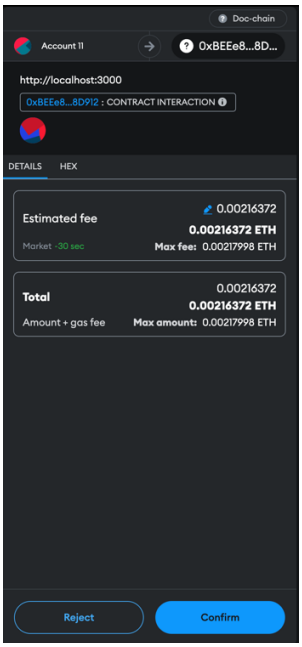


D. InterPlanetary File System (IPFS) Integration :

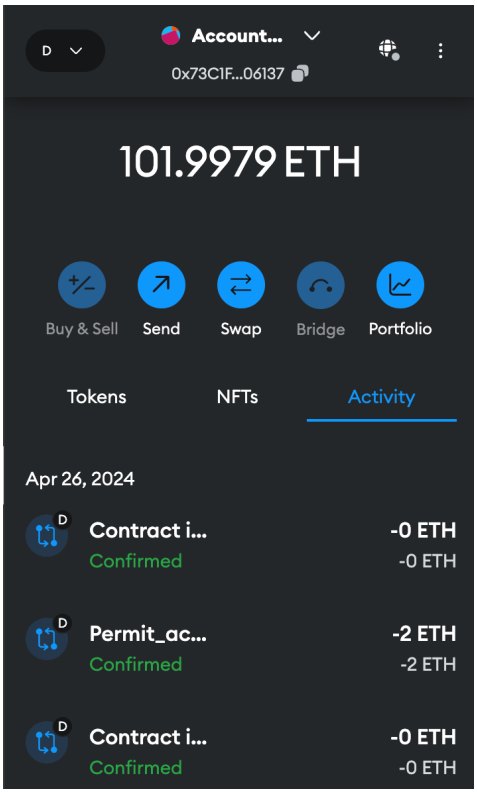


E. Metamask Wallet Transactions:

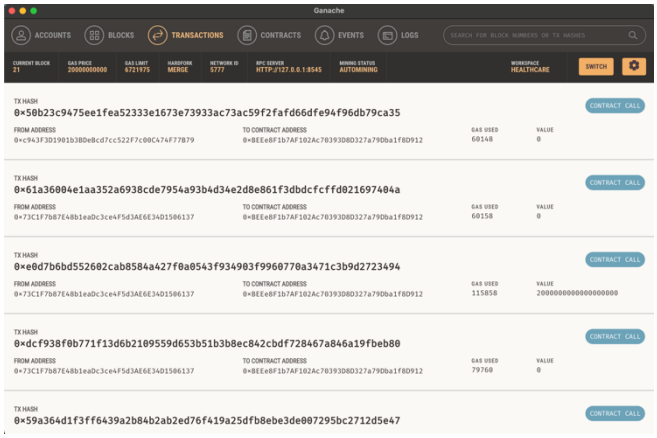
A. Transaction confirmation :



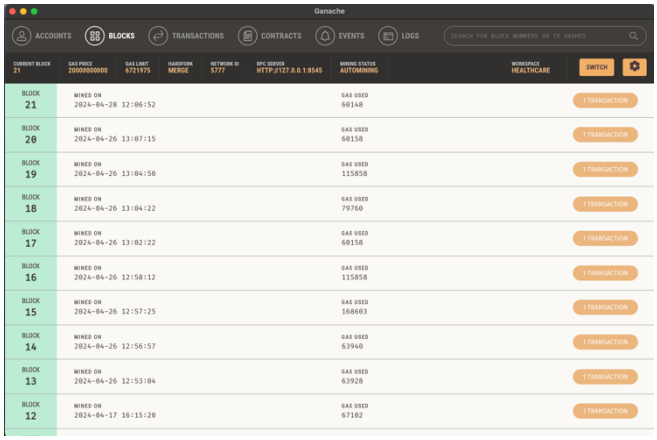
B. Account Dashboard :



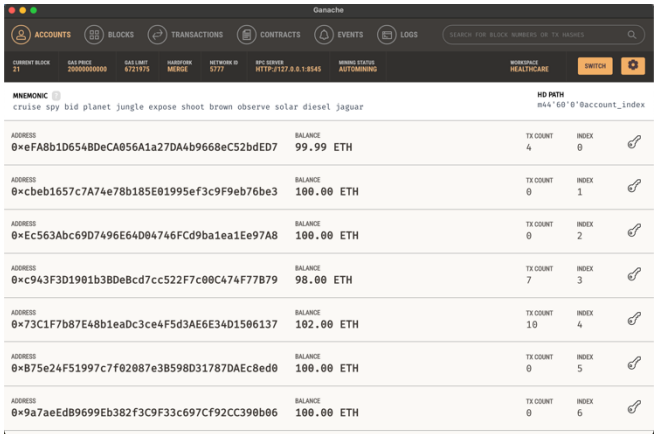
F. Transaction Details :



G. Mined Blocks in the Blockchain :



H. Accounts Dashboard in Ganache :



V. Proposed Algorithm

A. MAIN AGENT SOLODITY FILE

```
pragma solidity ^0.8.1;

contract Agent {

    struct patient {
        string name;
        uint age;
        address[] doctorAccessList;
        uint[] diagnosis;
        string record;
    }

    struct doctor {
        string name;
        uint age;
        address[] patientAccessList;
    }

    uint creditPool;

    address[] public patientList;
    address[] public doctorList;

    mapping (address => patient) patientInfo;
    mapping (address => doctor) doctorInfo;
    mapping (address => address) Empty;
    // might not be necessary
    mapping (address => string) patientRecords;


    function add_agent(string memory _name, uint _age, uint
_designation, string memory _hash) public returns(string
memory){
        address addr = msg.sender;

        if(_designation == 0){
            patient memory p;
            p.name = _name;
            p.age = _age;
            p.record = _hash;
            patientInfo[msg.sender] = p;
            patientList.push(addr)-1;
            return _name;
        }
        else if (_designation == 1){
            doctorInfo[addr].name = _name;
            doctorInfo[addr].age = _age;
            doctorList.push(addr)-1;
            return _name;
        }
        else{
            revert();
        }
    }


    function get_patient(address addr) view public returns
(string memory , uint, uint[] memory , address, string
memory ){
        return (patientInfo[addr].name, patientInfo[addr].age,
patientInfo[addr].diagnosis, Empty[addr],
patientInfo[addr].record);
    }
```

```
    function get_doctor(address addr) view public returns
(string memory , uint){
        return (doctorInfo[addr].name, doctorInfo[addr].age);
    }
    function get_patient_doctor_name(address paddr, address
daddr) view public returns (string memory , string memory
){
        return
(patientInfo[paddr].name,doctorInfo[daddr].name);
    }

    function permit_access(address addr) payable public {
        require(msg.value == 2 ether);

        creditPool += 2;

        doctorInfo[addr].patientAccessList.push(msg.sender)-
1;
        patientInfo[msg.sender].doctorAccessList.push(addr)-
1;

    }

    //must be called by doctor
    function insurance_claim(address paddr, uint _diagnosis,
string memory  _hash) public {
        bool patientFound = false;
        for(uint i =
0;i<doctorInfo[msg.sender].patientAccessList.length;i++){

if(doctorInfo[msg.sender].patientAccessList[i]==paddr){
                msg.sender.transfer(2 ether);
                creditPool -= 2;
                patientFound = true;

            }

        }
        if(patientFound==true){
            set_hash(paddr, _hash);
            remove_patient(paddr, msg.sender);
        }else {
            revert();
        }

        bool DiagnosisFound = false;
        for(uint j = 0; j <
patientInfo[paddr].diagnosis.length;j++){
            if(patientInfo[paddr].diagnosis[j] ==
_diagnosis)DiagnosisFound = true;
        }
    }

    function remove_element_in_array(address[] storage
Array, address addr) internal returns(uint)
    {
        bool check = false;
        uint del_index = 0;
        for(uint i = 0; i<Array.length; i++){
            if(Array[i] == addr){
                check = true;
                del_index = i;
            }
        }
        if(!check) revert();
        else{
            if(Array.length == 1){
                delete Array[del_index];
            }
            else {
                Array[del_index] = Array[Array.length - 1];
                delete Array[Array.length - 1];

            }
            Array.length--;
        }
    }

    function remove_patient(address paddr, address daddr)
public {

remove_element_in_array(doctorInfo[daddr].patientAccess
List, paddr);

remove_element_in_array(patientInfo[paddr].doctorAccess
List, daddr);
    }

    function get_accessed_doctorlist_for_patient(address
addr) public view returns (address[] memory )
    {
        address[] storage doctoraddr =
patientInfo[addr].doctorAccessList;
        return doctoraddr;
    }
    function get_accessed_patientlist_for_doctor(address
addr) public view returns (address[] memory )
    {
        return doctorInfo[addr].patientAccessList;
    }


    function revoke_access(address daddr) public payable{
        remove_patient(msg.sender,daddr);
        msg.sender.transfer(2 ether);
        creditPool -= 2;
    }

    function get_patient_list() public view returns(address[]
memory ){
        return patientList;
    }

    function get_doctor_list() public view returns(address[]
memory ){
        return doctorList;
    }

    function get_hash(address paddr) public view
returns(string memory ){
        return patientInfo[paddr].record;
    }
```

```
    function set_hash(address paddr, string memory _hash)
internal {
        patientInfo[paddr].record = _hash;
    }

}
```

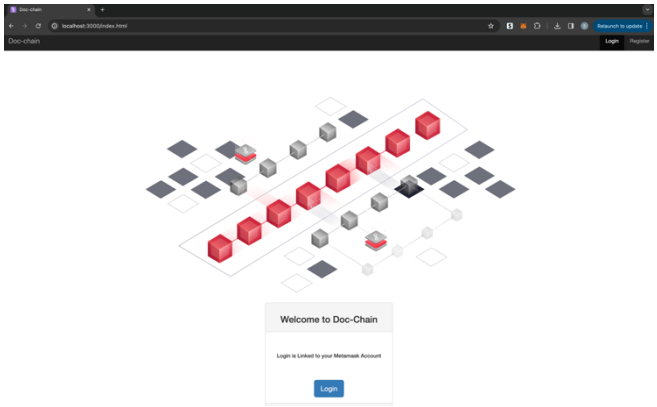
VI. Security and Privacy

In Doc-chain, stringent security measures are implemented to protect patient privacy and ensure data integrity. Robust encryption algorithms are employed to securely encrypt medical records stored on IPFS, with encryption keys managed by patient smart contracts, allowing only authorized access. Smart contracts enforce granular access control, enabling patients to manage access permissions dynamically and limiting access to authorized doctors. Comprehensive auditing and logging mechanisms record all transactions and interactions on the blockchain, facilitating transparency and accountability while detecting unauthorized access attempts. Moreover, Doc-chain adopts a data minimization approach to reduce the storage of personally identifiable information, adhering to regulatory standards such as HIPAA. These security measures collectively establish Doc-chain as a secure and compliant platform for decentralized medical records management.

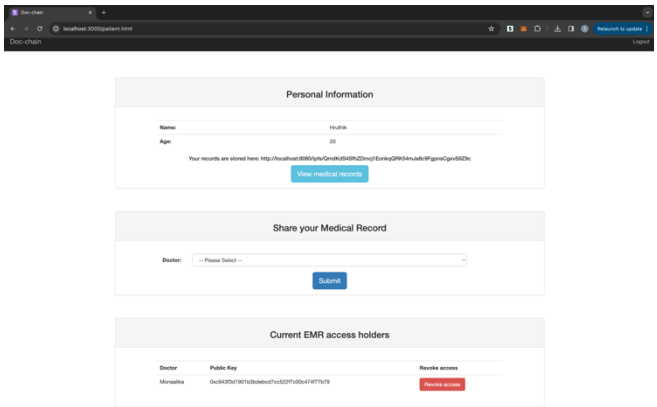
VII. Results

Output Screenshots :

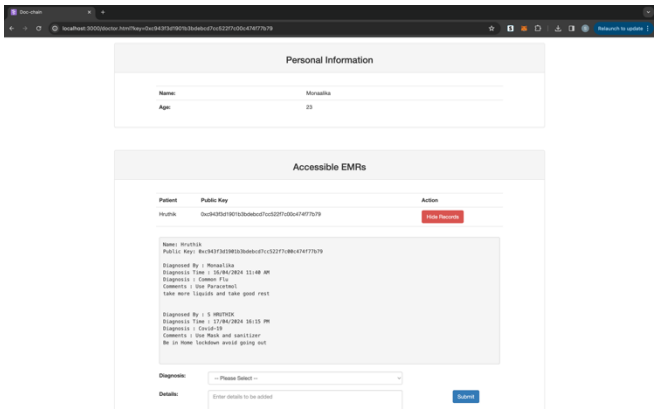
A. Login Page :



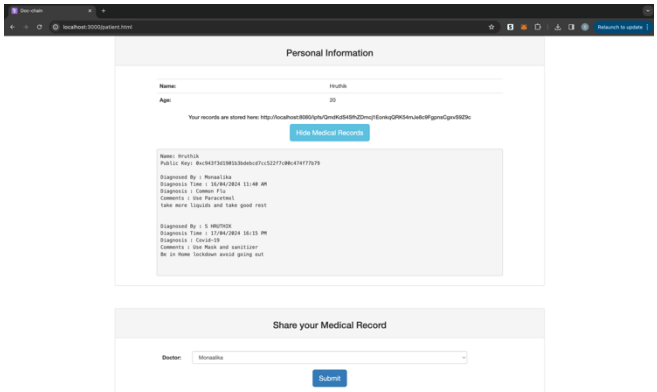
B. Patient portal :



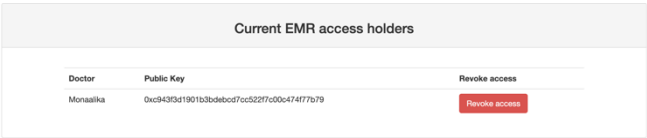
C. Doctor Portal :



D. Patient Records stored in Decentralised Storage and Public Key Overview :



E. Revoke Access to Medical Record :



VIII. Conclusion

In this paper, we presented Doc-chain, a decentralized medical record management system leveraging Ethereum smart contracts and IPFS to address challenges in healthcare data management. The proposed system offers significant advantages in terms of data security, privacy, accessibility, and user control. Through the implementation and evaluation of Doc-chain, several key findings and contributions have been identified.

The performance evaluation of Doc-chain demonstrated promising results in terms of system efficiency and reliability. Transaction throughput was measured to be 70.9 #x/s, indicating the platform's ability to handle a high volume of access requests and record updates. Response time for user interactions was 50 ms, ensuring a seamless and responsive user experience. showcasing efficient resource allocation and utilization within the decentralized network.
with the Doc-chain's usability, functionality, and security features. Patients has the ability to revoke access to their medical records at any time, enhancing their data privacy and autonomy. Doctors has the easy system intuitive and efficient for accessing patient records securely.

In conclusion, Doc-chain represents a significant advancement in decentralized healthcare data management, offering enhanced security, privacy, and user control compared to traditional centralized systems. Future work will focus on scalability enhancements, interoperability with existing healthcare infrastructure, and further evaluation in real-world healthcare settings. By addressing the challenges of data privacy and accessibility, Doc-chain contributes to the evolution of healthcare systems towards more secure and patient-centric solutions.

REFERENCES

[1] Zaabar, B., Cheikhrouhou, O., Jamil, F., Ammi, M., & Abid, M. (2021). HealthBlock: A secure blockchain-based healthcare data management system. *Computer Networks*, *200*, 108500.

[2] Al Mamun, A., Azam, S., & Gritti, C. (2022). Blockchain-based electronic health records management: a comprehensive review and future research direction. *IEEE Access*, *10*, 5768-5789.

[3] Ramani, V., Kumar, T., Bracken, A., Liyanage, M., & Ylianttila, M. (2018, December). Secure and efficient data accessibility in blockchain based healthcare systems. In *2018 IEEE Global Communications Conference (GLOBECOM)* (pp. 206-212). IEEE.

[4] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

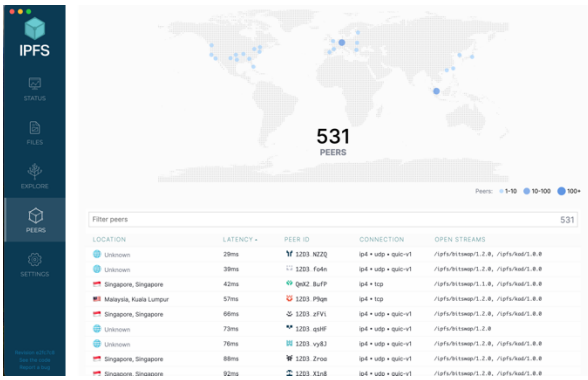
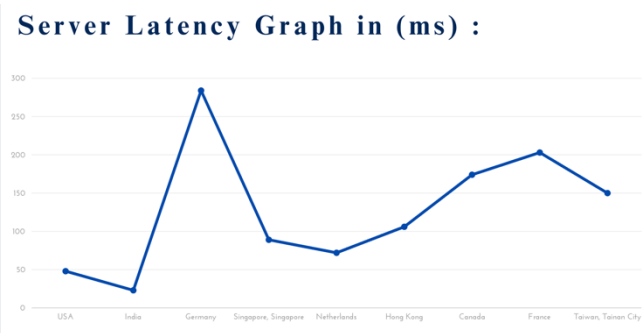
[5] Tanwar, S., Parekh, K., & Evans, R. (2020). Blockchain-based electronic healthcare record system for healthcare 4.0 applications. *Journal of Information Security and Applications*, *50*, 102407.

[6] Chelladurai, U., & Pandian, S. (2022). A novel blockchain based electronic health record automation system for healthcare. *Journal of Ambient Intelligence and Humanized Computing*, *13*(1), 693-703.

[7] Attaran, M. (2022). Blockchain technology in healthcare: Challenges and opportunities. *International Journal of Healthcare Management*, *15*(1), 70-83.

[8] Shahnaz, A., Qamar, U., & Khalid, A. (2019). Using blockchain for electronic health records. *IEEE access*, *7*, 147782-147795.

GitHub Link of DApp : https://github.com/Hruthik9/Doc-Chain

Graphs link : https://www.canva.com/design/DAGDr5GZFec/ebXyJeyyGvqrqUWA7FOGgg/edit

IPFS Server peers and its Latency to access those Servers :